Zavarovalnica Horica

RSO 2026/27

Naslov projekta

Zavarovalnica Horica - zavarovalnica, pri katerem bodo agenti umetne inteligence pomagali pri obdelavi zahtevkov, tvegane odločitve pa bodo potrjevali zaposleni.

Podatki o projektu

Ime skupine:	Zavarovalnica Horica
Številka projektne skupine:	3
Člani skupine:	Sandi Cejan, Miha Maksimiljan Bertoncelj, Kristjan Brataševec

Kratek opis projekta

Pri obravnavi odškodninskih zahtevkov je treba preveriti veljavnost zavarovalne police, zbrati dokazila in oceniti, ali je zahtevek upravičen. Ker je del postopka ponavljajoč, ga je smiselno do neke mere avtomatizirati, pri zahtevnejših odločitvah pa ohraniti človeški nadzor. V projektu bomo razvili spletno aplikacijo, v kateri bodo stranke prijavile škodo, priložile dokumente (npr. slike, potrdila, račune ...) in spremljale stanje zahtevka. Sistem bo sestavljen iz šestih mikrostoritev. Dva logična agenta znotraj ene izmed njih bosta prek obstoječih programskih vmesnikov zbrala podatke, preverila izpolnjenost pogojev in pripravila predlog nadaljnje obravnave. Preproste zahtevke bo mogoče zaključiti samodejno po vnaprej določenih pravilih, zahtevke z večjim tveganjem pa bo pregledal zaposleni. Aplikacijo bomo razvili v okolju .NET in namestili na oblačno platformo Microsoft Azure.

Ogrodje in razvojno okolje

Zaledni del bomo razvijali v programskem jeziku **C#** z uporabo tehnologije **.NET**. Za uporabniški vmesnik bomo uporabili **React in TypeScript**, kateri bo omogočal ločen pogled glede na to, ali je uporabnik spletišča stranka ali zaposleni. Zahteve iz uporabniškega vmesnika bo do mikrostoritev bomo usmerjali z uporabo prehoda API Gateway.

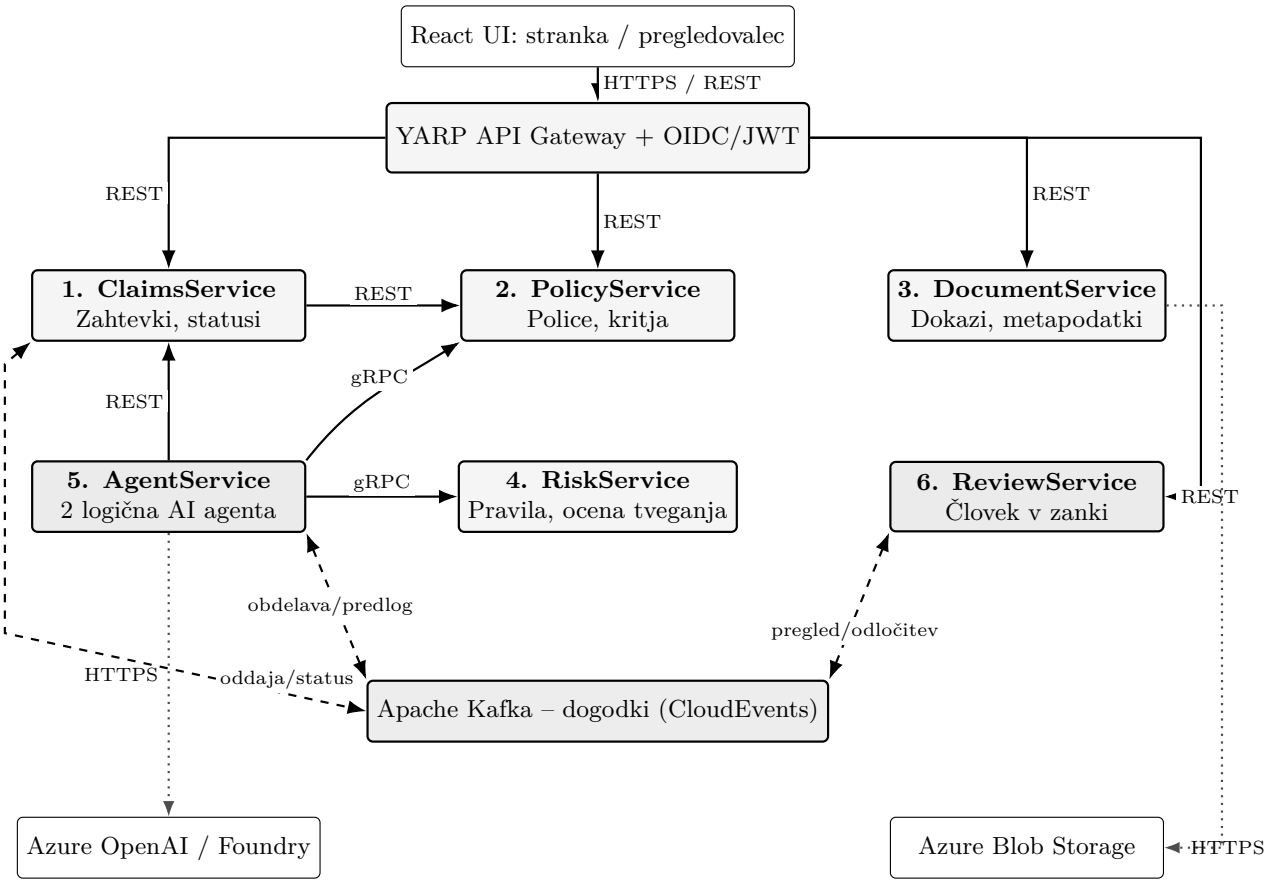
Mikrostoritve bodo med seboj komunicirale prek **REST** in **gRPC**, za dogodke, kot sta oddaja zahtevka ali zaključek pregleda, pa bomo uporabili orodje **Apache Kafka**. Vsaka storitev bo imela svojo logično podatkovno zbirko PostgreSQL (na Azure), priložene fotografije in dokumente pa bomo shranjevali v Azure Blob Storage. Za razvoj agentov načrtujemo uporabo modelov LLM, kot so Google Gemini ali Grok oz. orodij računalniškega vida za namene obdelave slik, kot je **Azure Vision in Foundry Tool**.

Razvijali bomo v Visual Studio oziroma Visual Studio Code, za lokalno okolje pa bomo uporabili **Docker Compose**. Vsebniške slike bomo objavljali na Docker Hub, storitve namestili na Azure **AKS** postopke CI/CD pa izvajali prek GitHub Actions, kjer bomo hranili izvorno kodo projekta. Avtentikacijo bomo uredili z OAuth 2.0 oziroma OpenID Connect (z uporabo Keycloak). Za spremljanje delovanja bomo po potrebi uporabili OpenTelemetry, Prometheus in Grafano, za testiranje pa xUnit in Testcontainers.

Za skupno delo načrtujemo GitHub organizacijo `zavarovalnica-horica` ter repozitorija **horica-platform** za izvorno kodo in **horica-infra** za konfiguracijo namestitve ter dokumentacijo.

Med razvojem bo lahko prišlo tudi do sprememb (kot so uporaba drugega orodja, oz. pristopa, npr. za namene avtentikacije).

Shema arhitekture



Vsaka izmed šestih mikrostoritev ima lastno logično PostgreSQL zbirko.

Slika 1: Začetna arhitektura Zavarovalnice Horica

Arhitekturo bomo razdelili na šest poslovnih mikrostoritev, od katerih bo vsaka skrbela za svoj del obravnave zahtevka. Poleg teh so tudi spletni vmesnik, API Gateway, Kafka, podatkovne zbirke in druge storitve kot podporne komponente.

Za pridobivanje podatkov bomo uporabili sinhrone klice REST oziroma gRPC. Ker lahko obravnava zahtevka traja tudi več ur ali dni, bomo spremembe stanja posredovali asinhrono prek Kafke. Ob oddaji zahtevka bo na primer `ClaimsService` objavila dogodek `claim.submitted.v1`, `AgentService` pa po analizi `claim.recommendation.created.v1`. Če bo potreben pregled zaposlenega, bo `ReviewService` prejela opravilo prek dogodka `review.requested.v1` in po odločitvi objavila `review.completed.v1`.

Vsaka odločitev bo zabeležena, da bomo lahko preverili, kako je zahtevek prišel do končnega stanja. Za začetno namestitev Kafke na AKS predvidevamo uporabo Strimzi.

Seznam funkcionalnosti mikrostoritev

1. **ClaimsService - upravljanje zahtevkov.** Omogočala bo oddajo zahtevka, pregled njegovega stanja in zgodovine obravnave. Podatke bo izpostavljala prek REST API-ja kot npr. `POST /claims` in `GET /claims/{id}`. Ob spremembah bo objavljala dogodke v Kafka. Ta storitev bo tudi odgovorna za končno stanje zahtevka.
2. **PolicyService - zavarovalne police.** Hranila bo podatke o policah, veljavnosti in vrstah kritja. Prek REST ali gRPC bo drugim storitvam omogočala preverjanje, ali zavarovanje krije prijavljeno škodo.
3. **DocumentService - dokazila.** Omogočala bo nalaganje fotografij, računov in drugih prilog, njihovo preverjanje ter shranjevanje v Azure Blob Storage. Druge storitve bodo prek API-ja pridobile podatke o dokazilih. Ob dodajanju dokumenta bo lahko objavila dogodek `document.added.v1`.
4. **RiskService - poslovna pravila in tveganje.** Preverjala bo pogoje kritja, višino zahtevka, popolnost dokumentacije in druge določene dejavnike. Prek gRPC bo vrnila oceno tveganja in razloge za ročni pregled.

Pravila bodo ločena od jezikovnega modela, da bodo rezultati preverljivi.

5. **AgentService - agentna obravnava.** Vključevala bo dva logična agenta. Prvi bo prek API-jev zbral podatke o zahtevku, polici in dokazilih, drugi pa bo na podlagi preverjenih podatkov ter rezultata `RiskService` pripravil povzetek in predlog. Rezultat bo posredovan z dogodkom `claim.recommendation.created.v1`. Če bo zahtevek tvegan, agent ne bo slednjega samodejno potrjeval.
6. **ReviewService - pregled zaposlenega.** Ustvarjala bo opravila za zahtevke, pri katerih je potreben človeški pregled. Zaposleni bo lahko zahtevek odobril, zavrnil ali zahteval dopolnitev. Storitev bo hranila razlog in čas odločitve ter objavila dogodek `review.completed.v1`.

V začetni različici bomo samodejno odobrili le zahtevke z veljavno polico, popolnimi dokazili, zneskom do 300€ in oceno tveganja največ 0,20. Pri višjih zneskih, nejasnostih ali predlogu zavrnitve bo potreben pregled zaposlenega. Meje so okvirne, tako kot končna namestitev, se bo med razvojem še lahko spremenila.

Primeri uporabe

UC-1: Pregled police. Stranka preveri veljavnost zavarovanja in obseg kritja.

UC-2: Oddaja zahtevka. Stranka prijavi škodni dogodek in vnese zahtevani znesek.

UC-3: Dodajanje dokazil. Stranka priloži fotografije, račune ali druga dokazila.

UC-4: Spremljanje statusa. Stranka spremlja potek obravnave in morebitne zahteve za dopolnitev.

UC-5: Samodejna obravnava. Sistem preveri preprost zahtevek in ga po pravilih lahko odobri.

UC-6: Ročni pregled. Zaposleni pregleda zahtevek z večjim tveganjem in sprejme odločitev.

UC-7: Ponovna analiza. Po dopolnitvi dokazil sistem ponovno preveri zahtevek.

UC-8: Pregled zgodovine. Pooblaščeni zaposleni pregleda odločitve in spremembe zahtevka.

Kompleksnejši primer uporabe: UC-6 – zahtevek s človeško potrditvijo

Za prikaz sodelovanja več mikrostoritev bomo uporabili zahtevek v vrednosti 1.200€. Ker presega prag samodejne odobritve, bo potrebna odločitev zaposlenega.

1. korak: Stranka odda zahtevek ter priloži fotografijo in račun. `ClaimsService` shrani zahtevek, `DocumentService` pa dokazila.
2. korak: `ClaimsService` prek Kafke objavi dogodek `claim.submitted.v1`.
3. korak: `AgentService` prevzame dogodek. Prvi agent prek API-jev pridobi podatke iz `ClaimsService`, `PolicyService` in `DocumentService`.
4. korak: `RiskService` preveri poslovna pravila in izračuna oceno tveganja. Drugi agent na tej podlagi pripravi povzetek in predlog.
5. korak: Agent objavi predlog v Kafko. Zaradi višjega zneska `ClaimsService` zahtevek preusmeri v ročni pregled z dogodkom `review.requested.v1`.
6. korak: `ReviewService` ustvari opravilo. Zaposleni pregleda dokazila in predlog ter zahtevek odobri, zavrne ali zahteva dopolnitev.
7. korak: `ReviewService` objavi `review.completed.v1`; `ClaimsService` posodobi stanje in shrani razlog odločitve.
8. korak: Stranka novo stanje vidi v spletnem vmesniku.